UCS 503 – SOFTWARE ENGINEERING

PROPERTY PULSE

Real Estate Platform with ML Price Predictor

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Submitted by:

1024030108 Guntaas Singh
1024030112 Haneesh
1024030538 Dhruv Rajput
102303133 Atiksh Gupta

Group: 3C14, BE Third Year, COE

Submitted to:
Ms. Anushka

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology

October 2026

Contents

1 Introduction **3**

1.1 Purpose **3**

1.2 Intended Audience and Reading Suggestions **3**

1.3 Project Scope **3**

Figure 1.1: Entire Work-flow **4**

2 Overall Description **5**

2.1 Product Perspective **5**

2.2 User Classes and Characteristics **5**

Figure 2.1: Type of Users **5**

2.3 Product Functions **6**

2.4 Operating Environment **6**

2.5 Design Activities and Architectural Models **7**

Figure 2.2: Activity Diagram — Buyer Lifecycle **7**

Figure 2.3: Activity Diagram — Seller Lifecycle **8**

Figure 2.4: System Use Case Diagram **9**

Figure 2.5: Class Diagram & Relational Schema **10**

Figure 2.6: Sequence Diagram — Seller Submission **11**

Figure 2.7: Sequence Diagram — Buyer Browsing & Save **12**

3 System Features **13**

3.1 Description and Priority **13**

3.2 Functional Requirements (Module Breakdown) **13**

4 Other Nonfunctional Requirements **14**

4.1 Performance Requirements **14**

4.2 Security Requirements **14**

4.3 Software Quality Attributes **14**

4.4 Business Rules & Evaluation Metrics **14**

5 Other Requirements **15**

5.1 Operational Constraints **15**

5.2 Future System Evolution **15**

1 Introduction

1.1 Purpose

The real estate market currently struggles with fragmented, unverified property listings, opaque pricing mechanisms, and slow buyer-seller transaction coordination. Buyers cannot easily determine whether an asking price reflects authentic market valuation, while sellers suffer from duplicate listings and lack of transparent price guidance.

PROPERTY PULSE (also referred to as *EstatePulse*) is an integrated, web-based real estate platform developed to eliminate market asymmetry. By embedding a Machine Learning pricing model directly into the property listing lifecycle and introducing an automated **Buyer Barometer**, the software empowers both buyers and sellers with objective pricing benchmarks, spam prevention, and a streamlined negotiation workflow.

1.2 Intended Audience and Reading Suggestions

This Software Requirements Specification (SRS) is prepared for course evaluators (Ms. Anushka), software architects, developers, testers, and project managers.

- **Academic Evaluators:** Review Chapter 1 (Objectives), Chapter 3 (Functional Specifications), and Chapter 4 (Empirical Evaluation Metrics).
- **Development Team:** Refer to Chapter 2 (System Architecture, Activity & Sequence Diagrams) and Chapter 3 (API Module specifications).
- **Quality Assurance Engineers:** Validate against Section 4 (Performance, Security, and Non-functional Quality Attributes).

1.3 Project Scope

Property Pulse establishes a comprehensive, end-to-end digital transaction ecosystem for three core user groups: **General Users**, **Buyers**, and **Sellers**.

When a seller submits a property listing with physical dimensions, age, city, and furnishing attributes, an automated ML Price Predictor computes an instant estimated fair market price ($\hat{y}$). Simultaneously, a dedicated Duplicate Checker queries the database to flag potential duplicate listings before insertion.

Buyers explore curated listings guided by the **Buyer Barometer**—a visual indicator classifying properties into *Great Deal*, *Fair Market Price*, or *Premium* based on mathematical variance between asking prices and model predictions. Interested buyers can negotiate terms via an in-app WhatsApp-style chat and schedule site visits by paying a nominal booking token fee through an integrated checkout modal.

The platform instruments and measures empirical software engineering metrics, most notably **Time-to-Appointment** ($T_{appt} = t_{confirmed} - t_{inquiry}$), aiming for a median duration ≤ 2 hours during pilot operations.

1.4 System Work-flow Overview

Figure 1.1 illustrates the high-level workflow of the PropertyPulse platform, showing the continuous interaction cycle between Buyers, Sellers, the Machine Learning service, and the System Evaluation Engine.

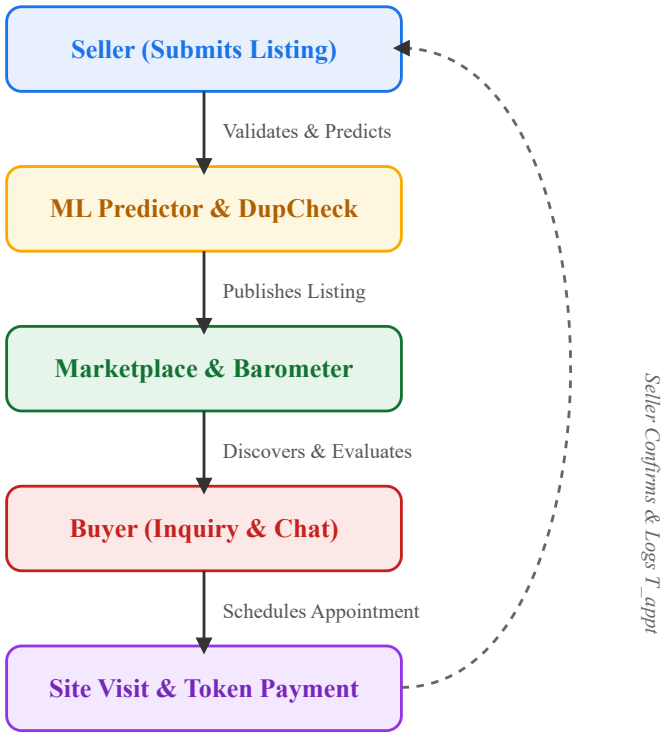


Figure 1.1: Entire System Work-flow

Every entity is interdependent. The seller's listing initiates the cycle; the machine learning service injects objective market context; the buyer makes data-driven negotiation decisions; and the evaluation engine tracks duration until confirmed physical inspection.

2 Overall Description

2.1 Product Perspective

PropertyPulse is an independent, 3-tier web platform engineered to replace manual and fragmented real estate ad postings. The data is centrally managed within an ACID-compliant relational schema, decoupling the client-side presentation layer from the predictive intelligence backend.

2.2 User Classes and Characteristics

The system identifies four primary user classes with defined permissions:

- **General User / Guest:** Unauthenticated visitor browsing listings with full search and Buyer Barometer visibility.
- **Buyer:** Authenticated buyer capable of sending messages, saving properties, and booking site visit appointments.
- **Seller:** Authenticated owner/broker with permissions to list properties, receive instant ML valuations, review duplicate alerts, chat with leads, and confirm visits.
- **Evaluator / Admin:** Course instructor or system administrator auditing regression diagnostics (R^2, MSE) and Time-to-Appointment metrics.

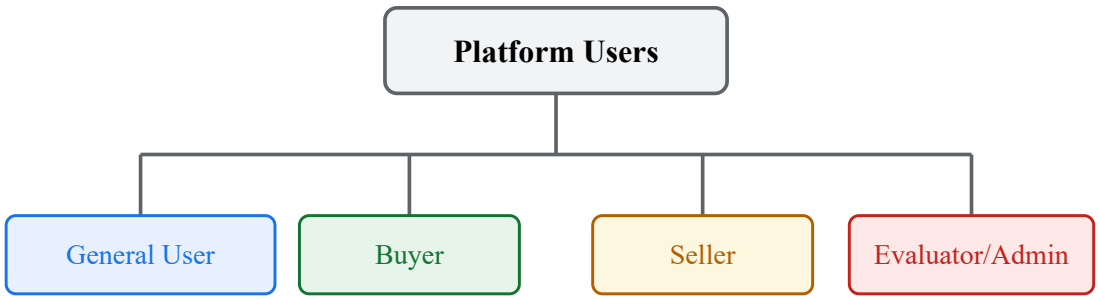


Figure 2.1: Type of Users

2.3 Product Functions

The core product functions encompass:

1. **Automated Valuation Engine:** Instant Multiple Linear Regression price estimation based on physical square footage, room counts, locality, age, and furnishing status.
2. **Buyer Barometer Visualization:** Real-time variance calculation labeling properties as *Great Deal*, *Fair Market Price*, or *Premium*.
3. **Duplicate Listing Prevention:** Lexical and structural candidate matching scanning existing database records.

4. **Property-Isolated Negotiation Chat:** Dedicated messaging threads eliminating fragmented communication.
5. **Site Visit Scheduling & Escrow Token:** Booking slot reservations accompanied by digital token fee settlements.
6. **Diagnostics Console:** Empirical reporting of Time-to-Appointment metrics and regression errors (R^2, MSE, RMSE).

2.4 Operating Environment

The software operates in any standard computing environment:

- **Operating System:** Windows 10/11, Ubuntu Linux 20.04+, macOS.
- **Runtime Stack:** Python 3.10+ (Flask 3.x, Scikit-learn 1.4+, NumPy, SQLite3).
- **Client Web Browsers:** Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge, and modern mobile browsers.

2.5 Design Activities and Architectural Models

2.5.1 Buyer Lifecycle Activity Diagram

Figure 2.2 details the workflow from a prospective buyer's perspective. The buyer initiates search and filter operations, inspects property details and the visual Buyer Barometer, saves the property or initiates real-time negotiation chat, and schedules a site visit via payment token checkout.

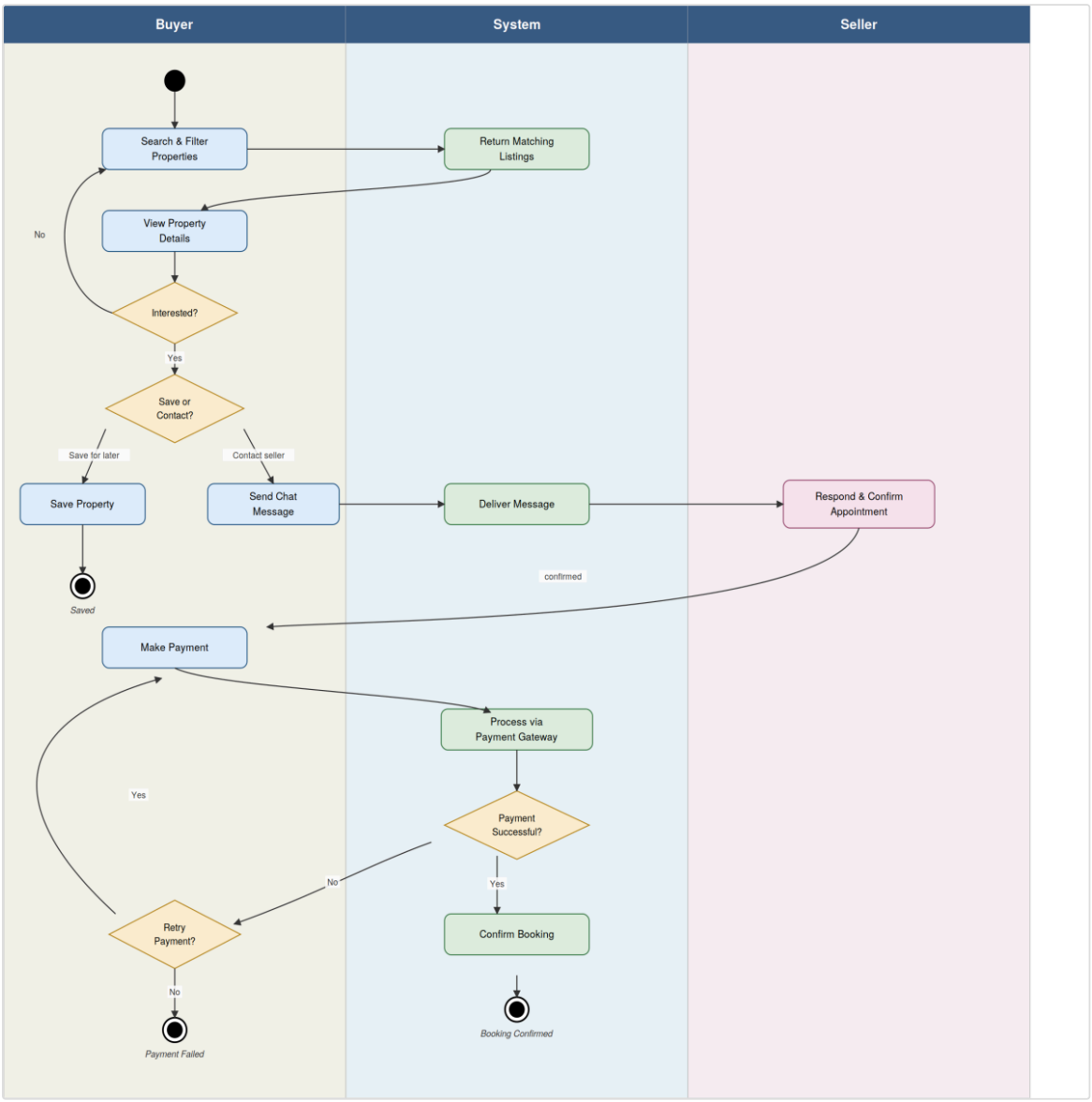


Figure 2.2: Activity Diagram — Buyer Lifecycle (Search, Save, Chat, and Site Visit Booking)

2.5.2 Seller Lifecycle Activity Diagram with ML Validation

Figure 2.3 depicts the seller's workflow. When a seller enters listing specifications, the system triggers the ML PricePredictor and DuplicateChecker before saving the listing. Upon receiving buyer inquiries, the seller negotiates and confirms the appointment slot, logging the confirmed timestamp.

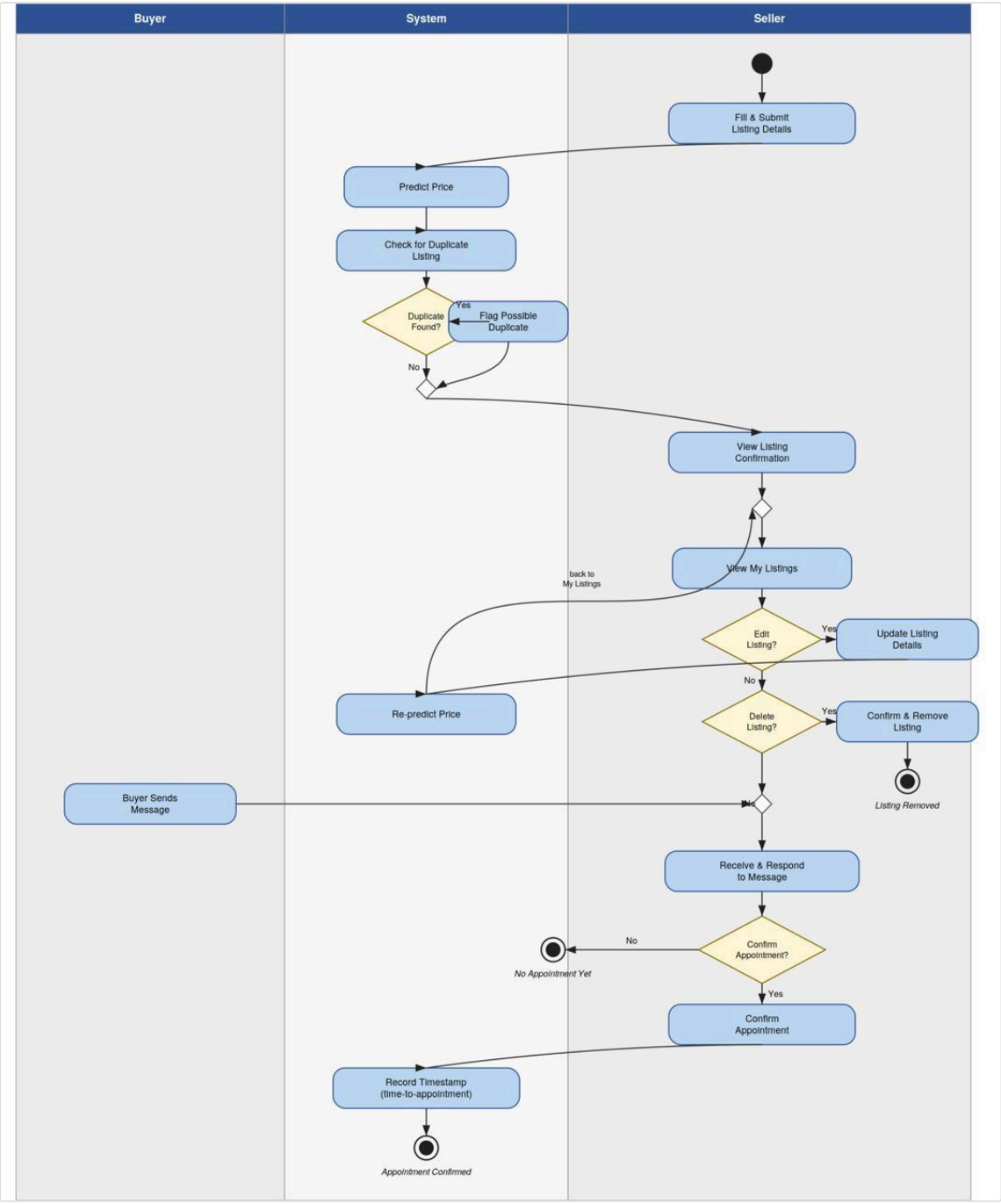


Figure 2.3: Activity Diagram — Seller Lifecycle with ML Price Prediction and Duplicate Check

2.5.3 System UML Use Case Diagram

Figure 2.4 specifies the functional interactions across all system actors: General User, Buyer, Seller, and the External Payment Gateway. Core use cases incorporate «include» dependencies for price prediction, duplicate checking, and payment confirmation.

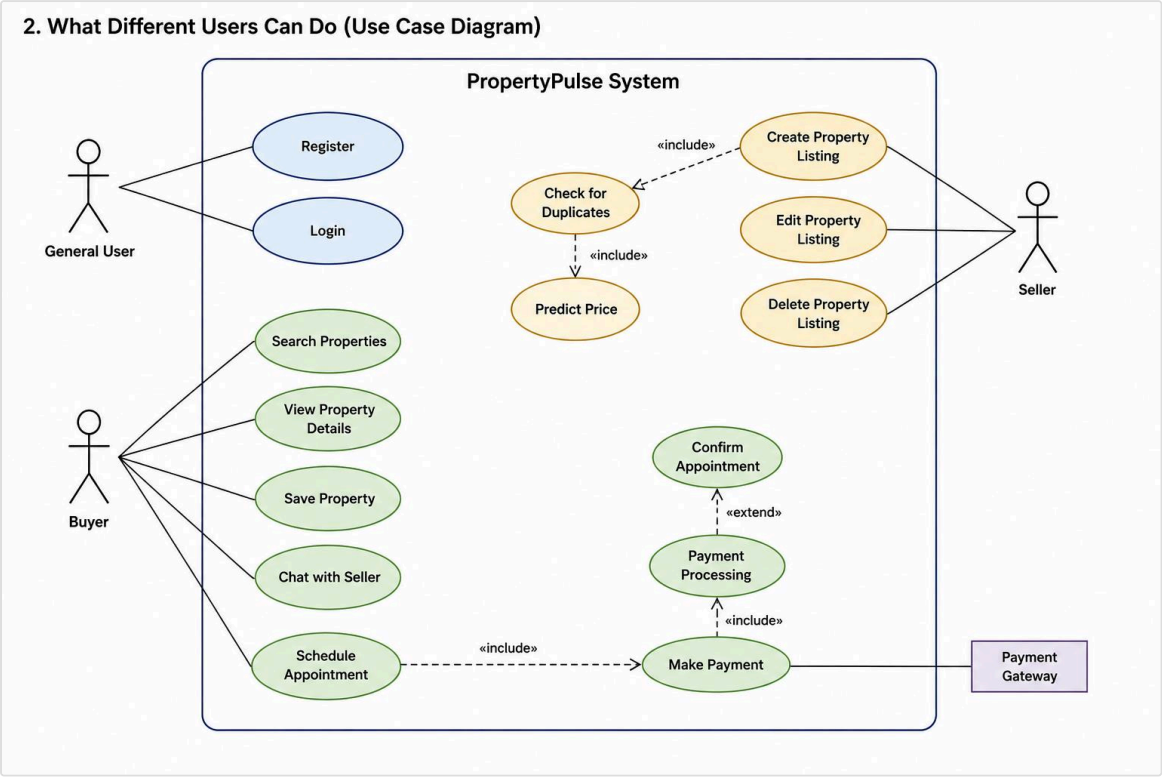


Figure 2.4: System Use Case Diagram across General Users, Buyers, Sellers, and Payment Gateway

2.5.4 UML Class Diagram and Relational Schema

Figure 2.5 defines the domain model and database architecture. Primary entities include `User`, `Property`, `SavedProperty`, `PropertyImage`, `Message`, `Appointment`, and `Payment`, with service delegation to `PricePredictor`.

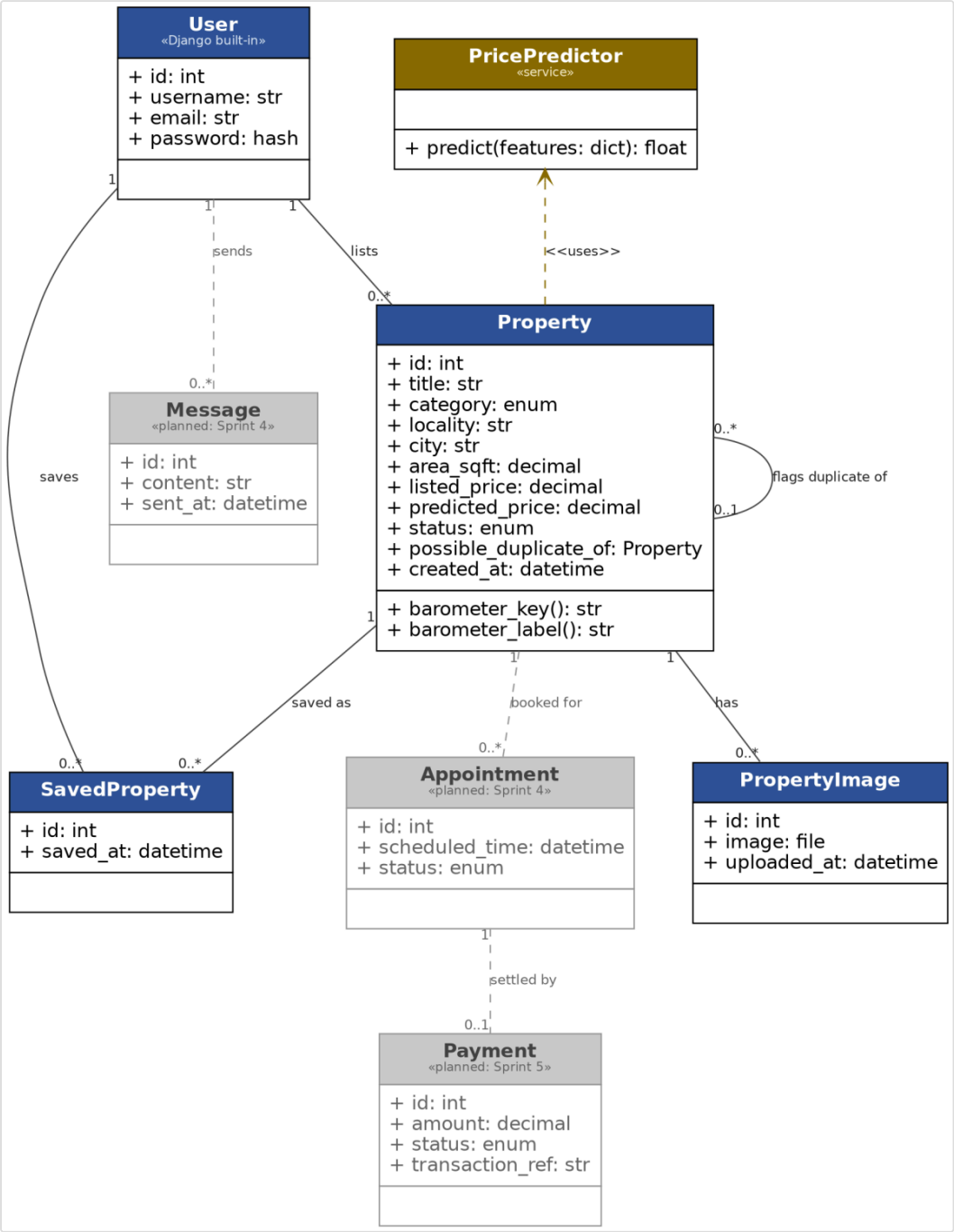


Figure 2.5: UML Class Diagram & Entity Relationships

2.5.5 Sequence Diagram — Seller Listing & ML Valuation

Figure 2.6 illustrates the chronological message exchange during property submission. The controller delegates feature vectors to the ML PricePredictor and verifies candidate duplicates prior to persistence.

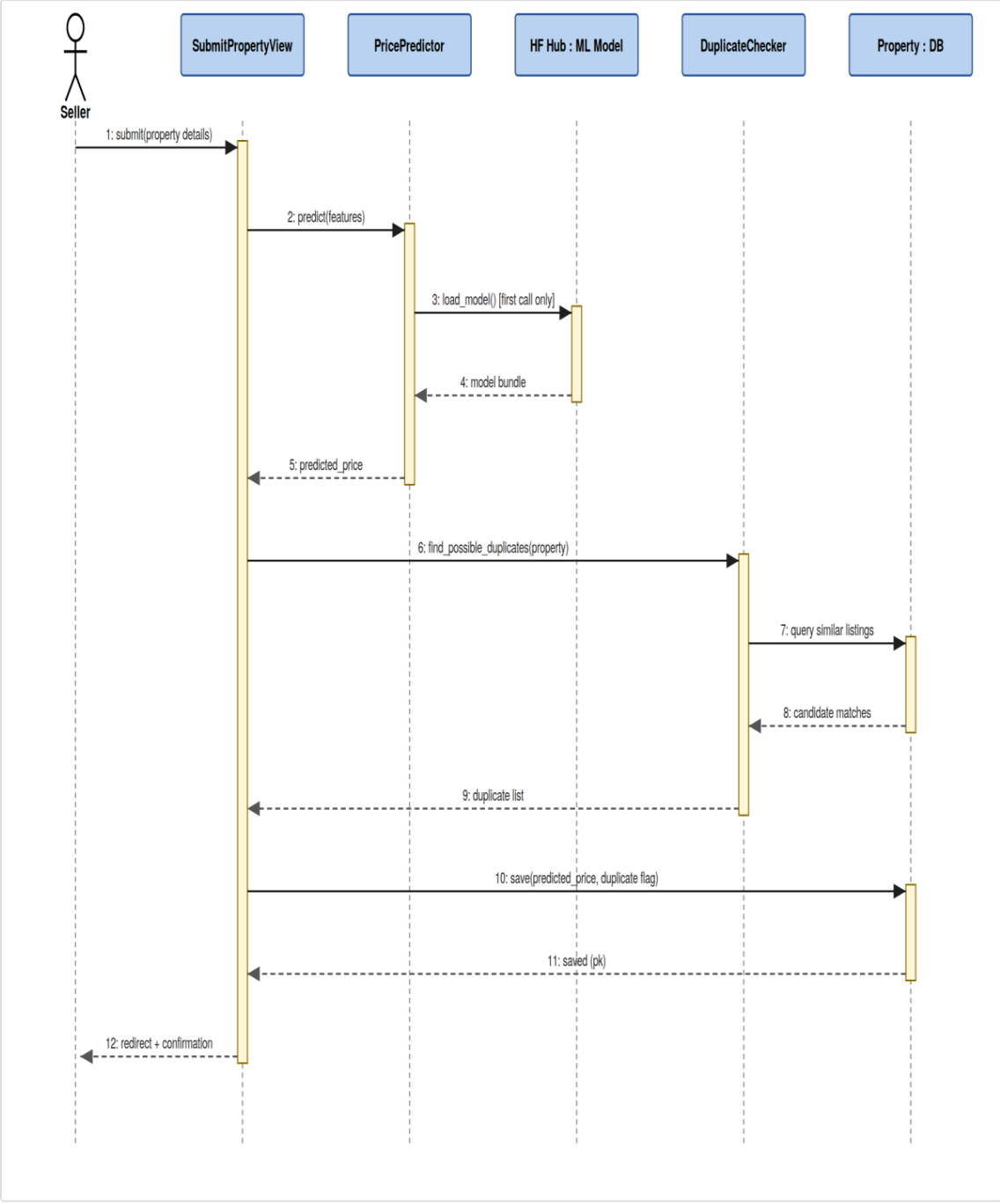


Figure 2.6: Sequence Diagram — Seller Listing Submission, ML Valuation, and Duplicate Check

2.5.6 Sequence Diagram — Buyer Browsing & Save

Figure 2.7 shows the sequence of interactions when a buyer queries active listings, filters by Barometer rating, and saves a selected property to their authenticated dashboard.

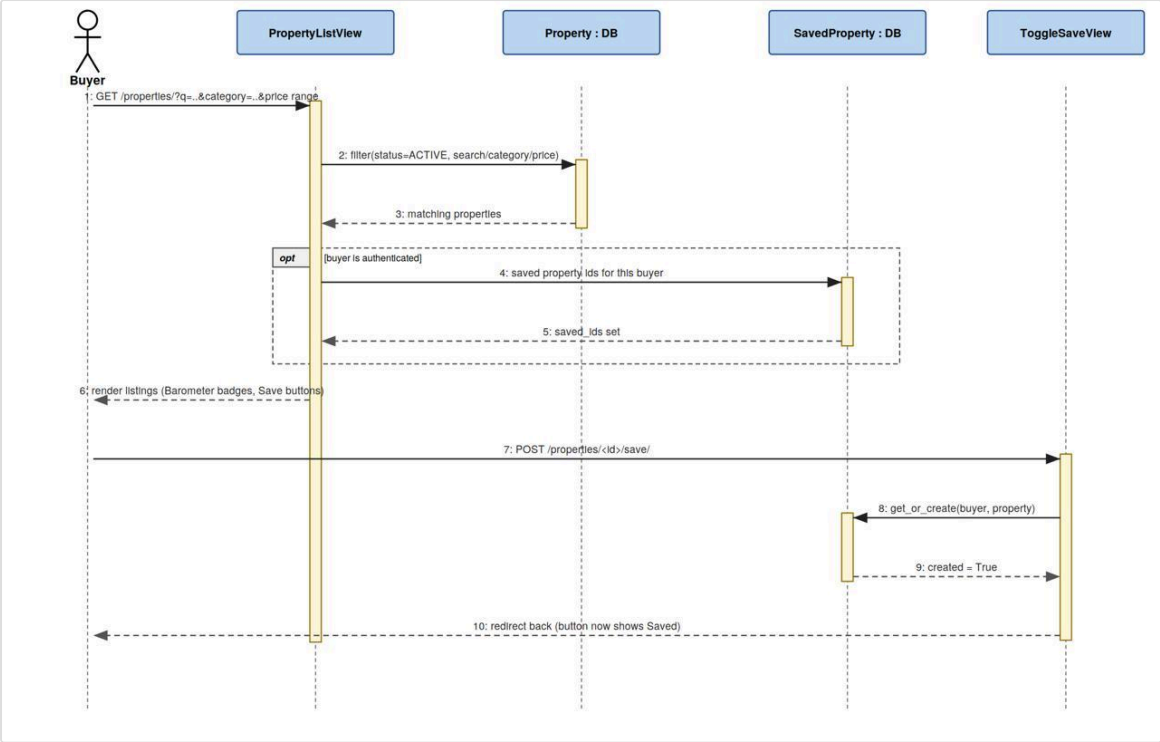


Figure 2.7: Sequence Diagram — Buyer Browsing, Filtering, and Property Save Operations

3 System Features

3.1 Description and Priority

The functional features are prioritized into three operational tiers:

Priority	Feature ID	Feature Name	Description
High (P1)	FEAT-01	ML Price Predictor	Computes fair market valuation using Multiple Linear Regression.
High (P1)	FEAT-02	Duplicate Listing Checker	Scans database on submission to flag matching listings in locality.
High (P1)	FEAT-03	Buyer Barometer Visualizer	Categorizes listings into Great Deal, Fair Value, or Premium.
High (P1)	FEAT-04	Negotiation Chat Engine	Property-isolated messaging threads between buyer and seller.
Medium (P2)	FEAT-05	Appointment & Token Payment	Booking slot calendar and token payment transaction logging.
Medium (P2)	FEAT-06	Role-Based Authentication	Seamless phone login with isolated Buyer and Seller views.
Low (P3)	FEAT-07	Diagnostics & Attribution	Live telemetry tracking Time-to-Appointment and model errors.

3.2 Functional Requirements (Module Breakdown)

Module 1: Property Listing & Duplicate Detection

- **FR-1.1:** The system shall accept property listings containing Title, Description, Category, City, Locality, Area (sq.ft), Bedrooms, Bathrooms, Age, Parking, Furnishing, and Asking Price.
- **FR-1.2:** System shall validate positive numeric values for Area and Price.
- **FR-1.3:** When title and locality are entered, the system shall invoke `POST /api/check-duplicates` to flag similar existing records.

Module 2: ML Price Prediction & Buyer Barometer

- **FR-2.1:** The system shall invoke `POST /api/predict` upon specification entry and return estimated price ($\hat{y}$) within 100ms.

- **FR-2.2:** Price deviation percentage shall be computed as $\Delta\% = ((P_{ask} - P_{pred}) / P_{pred}) imes 100$.
- **FR-2.3:** Properties shall display color-coded badges: *Great Deal* ($\Delta\% \le -10\%$), *Fair Market Price* ($-10\% < \Delta\% \le +10\%$), or *Premium* ($\Delta\% > +10\%$).

Module 3: In-App Chat & Site Visit Scheduling

- **FR-3.1:** Each property shall maintain a dedicated messaging thread identified by `property_id`.
- **FR-3.2:** The receipt of the initial buyer message shall instantiate `first_inquiry_timestamp`.
- **FR-3.3:** Buyers shall select visit dates and settle token fees via `POST /api/payments`.
- **FR-3.4:** Seller confirmation via `POST /api/appointments/<id>/confirm` shall record `confirmed_timestamp` and calculate `time_to_appointment_seconds`.[12]

4 Other Nonfunctional Requirements

4.1 Performance Requirements

- **ML Inference Latency:** Price estimation response time shall remain below 100 milliseconds.
- **Page Load Time:** Client SPA shall render initial interactive view in < 1.5 seconds.
- **Database Latency:** Search queries filtering by city and category shall execute in < 50 milliseconds.
- **Service Level Target:** Median Time-to-Appointment shall not exceed 2 hours during pilot evaluation.

4.2 Security Requirements

- **SQL Injection Prevention:** All database transactions must strictly utilize parameterized SQL statements.
- **XSS Mitigation:** All user inputs (descriptions, chat text) must be escaped before DOM injection.
- **Access Control:** Sellers may only edit or delete properties registered to their verified phone numbers.

4.3 Software Quality Attributes

- **Usability & Mobile Responsiveness:** Fluid CSS media queries ensuring full touch usability ($\ge 44 imes 44$ px) across mobile and desktop devices.
- **Reliability:** Minimum 99% uptime during evaluation sessions; client network errors handled gracefully via toast notifications.
- **Maintainability:** Clean 3-tier architectural separation between database (`database.py`), ML logic (`ml_model.py`), and API routing (`app.py`).

4.4 Business Rules & Evaluation Metrics

The predictive valuation engine implements Multiple Linear Regression:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Model accuracy is evaluated using Mean Squared Error (MSE) and Coefficient of Determination (R^2):

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

The primary software engineering metric, **Time-to-Appointment**, is defined as:

$$T_{appt} = t_{confirmed} - t_{first_inquiry}$$

5 Other Requirements

5.1 Operational Constraints

- **Zero Commercial Dependencies:** System operates entirely on local open-source libraries (Scikit-learn, Flask, SQLite) with no paid API keys required.
- **Memory Footprint:** Lightweight server footprint (< 120 MB RAM during active ML inference) ensuring compatibility with low-spec lab environments.
- **Time-to-Value Delivery:** Focus on robust engineering workflows, test automation, and measurable metrics rather than speculative algorithm research.

5.2 Future System Evolution

- **WebSocket Integration:** Replacing 3-second HTTP chat polling with bi-directional WebSockets (Flask-SocketIO) for instant message delivery.
- **Computer Vision Deduplication:** Integrating convolutional neural network perceptual hashing (pHash) to detect duplicate property image uploads.
- **Cloud CI/CD Deployment:** Automated GitHub Actions workflow deploying containers to cloud PaaS platforms.